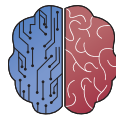




UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería de la Salud



INGENIERÍA
DE LA SALUD

**TFG del Grado en Ingeniería de la
Salud**

**Redes GAN para la
Generación y Ampliación de
Datos ECG en Conjuntos
Desequilibrados
Documentación Técnica**

Presentado por Paula Cuesta Asín
en Universidad de Burgos

8 de julio de 2025

Tutores: Olga Valencia García – Jaime Andrés Rincón
Arango

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
Apéndice A Documentación de usuario	13
A.1. Requisitos Software y Hardware para Ejecutar el Proyecto .	13
A.2. Instalación / Puesta en Marcha	15
A.3. Manuales y/o Demostraciones Prácticas	17
Apéndice A Documentación de usuario	19
A.1. Requisitos software y hardware para ejecutar el proyecto . .	19
A.2. Instalación / Puesta en marcha	21
Apéndice B Descripción de adquisición y tratamiento de datos	23
B.1. Características generales de los datos	23
Apéndice A Estudio experimental	33
A.1. Cuaderno de trabajo	34
Apéndice B Plan de Proyecto Software	35
B.1. Introducción	35
B.2. Planificación temporal	36

Índice de figuras

A.1. Organización de las tareas en el tiempo disponible.	9
B.1. Gráfico de barras que representa la diferencia de tamaño entre el dataset de entrenamiento y de validación. <i>Elaboración propia.</i> .	24
B.2. Histograma que representa el tamaño en número de muestras de las señales de entrenamiento. <i>Elaboración propia.</i>	27
B.3. Histograma que representa la duración en segundos de las señales del dataset de entrenamiento. <i>Elaboración propia.</i>	28
B.4. Histograma que representa el tamaño en número de muestras de las señales de validación. <i>Elaboración propia.</i>	29
B.5. Histograma que representa la duración en segundos de las señales de validación. <i>Elaboración propia.</i>	29
B.1. Diagrama de Gantt del Plan de Proyecto. <i>Elaboración propia</i> . .	40

Índice de tablas

A.1. Presentación de las secciones del Plan de Proyecto. <i>Elaboración propia</i>	2
B.1. Presentación de las secciones del Plan de Proyecto. <i>Elaboración propia</i>	36

Apéndice A

Plan de Proyecto Software

A.1. Introducción

Antes de iniciar un proyecto, especialmente uno relacionado con el desarrollo de software, es fundamental establecer una adecuada planificación. Esto responde a la propia naturaleza de esta disciplina, ya que constituye una actividad integral. Por ello, no requiere únicamente de competencias técnicas, sino que también incluye aspectos de gestión, evaluación y cumplimiento normativo. [Sommerville, 2015]

Esta tarea resulta clave incluso cuando se está trabajando en el ámbito académico, como ocurre en el caso de un Trabajo de Fin de Grado. Con este objetivo, el presente trabajo se ha realizado siguiendo los principios básicos de la Ingeniería del Software y de la gestión de proyectos tecnológicos, utilizando así un enfoque técnico y riguroso durante el mismo. [Basili, 2006]

Con este propósito, el plan de desarrollo que se presenta a continuación tiene como objetivo organizar, estructurar, dimensionar e incluso justificar las diferentes fases en las que se divide el proyecto. Para ello, se contemplan aspectos y estrategias tanto a nivel técnico como temporal, logístico, económico o legal, garantizando así su viabilidad y coherencia. Además, permite evitar complicaciones una vez empezado.

En este sentido, el siguiente Plan de Proyecto Software constituye un documento de referencia que plasma de forma clara y estructurada los factores determinantes relacionados con la gestión, asignación de recursos y desarrollo del proyecto, complementando así el enfoque teórico y técnico descrito en el cuerpo principal de la memoria.

Sección	Objetivos	Contenido
Planificación Temporal	Organizar el proyecto a lo largo de su longitud en el tiempo	Cronograma, fases y hitos
Planificación Económica	Presentar la inversión necesaria y la viabilidad financiera	Presupuesto, recursos y coste
Viabilidad Legal	Garantizar la legalidad, cumpliendo la normativa vigente	Normativas, licencias y aspectos regulatorios

Tabla A.1: Presentación de las secciones del Plan de Proyecto. *Elaboración propia*

Este apéndice se organiza en tres secciones fundamentales, que se detallan a continuación, y que se resumen en la tabla B.1, situada en la página 36.

A.2. Planificación temporal

La planificación temporal es una fase fundamental en el ámbito de la organización y la gestión eficaz de proyectos de cualquier tipo. [(PMI), 2021] Esta importancia radica, principalmente, en la necesidad del proyecto por establecer un marco de trabajo organizado y estructurado, el cual sea eficiente y de utilidad para la distribución temporal y sistemática de tareas. De esta forma, se asegura la finalización exitosa de las distintas actividades que se deben abordar a lo largo de su ciclo de vida en los plazos previstos para ello. Este hecho se hace tangible en el presente proyecto gracias a esta organización temporal seguida. [Sommerville, 2015]

Esta dimensión temporal adquiere un valor añadido en proyectos de naturaleza técnica o investigadora, especialmente si estos se basan en el desarrollo de software. Esto se debe a su naturaleza multifacética y a la estructuración de los mismos en multitud de fases iterativas de diversa índole, donde cada una de ellas presenta un nivel de complejidad, unos requisitos y una duración particular en función de las tareas en que se divida dicha fase o etapa.

Esta planificación, siempre y cuando sea adecuada al tiempo disponible para la realización del proyecto, facilita la consecución de los objetivos propuestos en los plazos establecidos para ello, permite anticiparse ante posibles problemas o desviaciones de las líneas de trabajo originales, optimiza el uso de los recursos y materiales disponibles y mejora el proceso de toma de decisiones informadas a lo largo del ciclo de vida del proyecto con el objetivo de evitar retrasos y asegurar una calidad óptima para el producto final.

En contextos o ámbitos académicos, como es el caso actual, una correcta y organizada planificación temporal resulta fundamental para el proyecto. Esto se debe a que dicha organización va a ayudar, e incluso garantizar, que el proyecto se desarrolle de forma coherente en el tiempo, alineándose su desarrollo en base al calendario académico docente propuesto por la institución educativa, a la vez que sigue los requisitos formales establecidos por la comisión de la titulación correspondiente. Por lo tanto, gracias a esta planificación se asegura un progreso coherente y bien documentado.

En el caso concreto del presente TFG, para lograr dicha organización eficaz del trabajo, se ha establecido una planificación iterativa basada en la metodología ágil Scrum. Esta se ha adaptado al contexto académico a través de pequeñas modificaciones, como la división del trabajo en sprints semanales o las revisiones periódicas por parte de los tutores. [Schwaber, Ken and Sutherland, Jeff,]

La planificación seguida a lo largo del presente proyecto, que utiliza la metodología ágil de Scrum, se organiza en una secuencia lógica de fases o etapas de una semana de duración, que se abordan a lo largo de varias semanas o *sprints* en las que se establecen tareas u objetivos concretos a completar durante cada uno de estos *sprints*.

Dichas tareas son también revisadas semanalmente en una reunión programada con los tutores durante la tarde del martes. A lo largo de la misma se efectúa la validación del trabajo realizado, planteando las posibles dudas y mejoras al respecto, y se planifican los hitos a lograr durante la semana en curso en cada momento.

Esta distribución temporal del Trabajo de Fin de Grado se puede observar en la imagen B.1, ubicada en la página 40. Esta representa un cronograma del presente trabajo a través de un diagrama de Gantt.

En este cronograma se muestran de forma clara y visual las actividades o tareas previstas para la realización completa del proyecto, su duración aproximada en el tiempo, las dependencias temporales que se establecen entre ellas, así como los objetivos o hitos que se pretenden alcanzar mediante la culminación de estas entregas intermedias.

Por lo tanto, si nos fijamos en este diagrama, lo podemos utilizar como una guía orientativa de los plazos que se deben cumplir a lo largo del desarrollo del trabajo, a la vez que nos sirve para evaluar de forma continua su progreso y cumplimiento (supervisando y comparando así el avance teórico del cronograma con el real) y para gestionar de una forma eficaz y

razonable la distribución de tareas a lo largo del tiempo disponible hasta su finalización.

A grandes rasgos, las tareas se pueden agrupar dando lugar a etapas o procesos. Estas reciben también el nombre de **Milestones** en el repositorio del presente trabajo, son las siguientes y se van a ir detallando a lo largo de esta sección:

1. Investigación y análisis acerca de la temática del proyecto
2. Declaración de objetivos a cumplir
3. Revisión bibliográfica inicial
4. Desarrollo e implementación técnica del trabajo
5. Validación técnica del sistema y evaluación de resultados
6. Documentación del proyecto y elaboración de la memoria

Cabe señalar que algunas de las etapas o *sprints* que se recogen en esta planificación, tales como la investigación preliminar, la definición de objetivos o la elaboración de la memoria, no se encuentran formando parte de la sección de Metodología incluida en la memoria del presente trabajo, ya que no se corresponden directamente con las fases técnicas o científicas del proyecto. No obstante, se trata de tareas complementarias a estas fases, las cuales resultan fundamentales para garantizar una ejecución completa y rigurosa del presente Trabajo de Fin de Grado (TFG).

Metodología de la planificación

Una vez introducida la planificación temporal del proyecto y la importancia de la misma para el correcto desarrollo de este, vamos a centrarnos en explicar la metodología concreta escogida para llevar a cabo dicha organización temporal.

La metodología utilizada para la organización y estructuración de esta planificación ha sido la metodología ágil Scrum, adaptando esta al entorno académico en que se desarrolla el presente TFG.

Este marco ágil de organización se utiliza en la mayor parte de los trabajos de desarrollo de software, llegando incluso a considerarse un *gold standard* por algunos integrantes de esta comunidad de programadores. [Schwaber, Ken and Sutherland, Jeff,]

Este tipo de metodología destaca por la organización iterativa e incremental del trabajo, dividiendo este en bloques o fases cortos y repetitivos, los cuales se conocen con el nombre de *sprints*. Esto permite que se puedan definir hitos clave o entregables intermedios a lo largo del proyecto, así como buscar un equilibrio entre las cargas de trabajo y los recursos disponibles, hechos que van a garantizar un desarrollo ordenado y eficiente.

Además, este enfoque de trabajo en pequeños fragmentos presenta numerosas ventajas, como la posibilidad de dar una respuesta rápida y flexible ante posibles cambios o desviaciones surgidas, la evaluación y revisión continua de los avances a lo largo de su desarrollo o la mejora continua y progresiva de las distintas versiones del producto final, el TFG en este caso. En definitiva, garantiza una correcta gestión del trabajo.

En los siguientes apartados se describen los principios fundamentales de la metodología Scrum aplicados al proyecto, los mecanismos utilizados para llevar a cabo las adaptaciones necesarias de este marco ágil de trabajo y las herramientas empleadas para realizar tanto la planificación, como el seguimiento y validación del trabajo en cada una de las iteraciones.

Adaptación de la metodología Scrum al entorno académico

Como ya se ha mencionado, en este proyecto, la metodología Scrum ha sido modificada ligeramente con el objetivo de adaptarla a la propia naturaleza del proyecto, teniendo en cuenta que este se ha desarrollado en el ámbito académico.

El modelo clásico de Scrum define roles específicos (Product Owner, Scrum Master, Development Team), reuniones (Daily Scrum, Sprint Planning, Sprint Review, Sprint Retrospective) y artefactos (Product Backlog, Sprint Backlog, Increment). Sin embargo, dado que este Trabajo de Fin de Grado no se ha desarrollado en un entorno empresarial con un equipo de desarrollo completo, sino de forma individual y bajo la supervisión de dos tutores, se han realizado las siguientes adaptaciones:

- **Estructura en sprints semanales:** El trabajo se ha organizado en ciclos de una semana de duración. Esta cadencia ha permitido un control detallado del avance y una carga de trabajo equilibrada, adaptada a la disponibilidad semanal del estudiante.
- **Roles adaptados:** El estudiante ha asumido un rol combinado de "equipo de desarrollo" "Product Owner", siendo responsable de la ejecución y la priorización de las tareas. Los tutores han desempeñado un

papel similar al de "Scrum Master", facilitando el proceso, resolviendo impedimentos y proporcionando guía técnica.

- **Reuniones de revisión y planificación semanales:** Todos los martes por la tarde se ha llevado a cabo una reunión con los tutores del proyecto. En estas sesiones, se revisaban los avances del sprint anterior (*Sprint Review*), se resolvían dudas, se analizaban posibles desviaciones y se establecían los objetivos y tareas prioritarias para el sprint siguiente (*Sprint Planning*). Estas reuniones han funcionado como el principal mecanismo de seguimiento y adaptación del plan.
- **Herramientas de gestión simplificadas:** Para la gestión del trabajo y la comunicación, se han utilizado herramientas accesibles y eficientes. El repositorio de **GitHub** ha servido como la principal plataforma para la gestión de versiones del código y la organización de **Milestones** y **Issues**, actuando como un *Sprint Backlog* rudimentario. Complementariamente, un **calendario** personal y acuerdos informales en las reuniones con los tutores han sustituido la necesidad de herramientas más complejas como Trello o Jira.
- **Entregables continuos:** Aunque no hay "incrementos" de software en el sentido estricto empresarial, cada sprint ha generado avances en el desarrollo técnico (código funcional, pruebas realizadas) y en la documentación del proyecto (apartados de la memoria, revisiones bibliográficas), lo que ha permitido una evaluación continua del progreso.

Estas adaptaciones han permitido aprovechar los beneficios de la agilidad de Scrum, como la flexibilidad, la transparencia y la mejora continua, dentro de las limitaciones y características del entorno académico de un Trabajo de Fin de Grado.

Etapas o Milestones del proyecto

El presente Trabajo de Fin de Grado se ha estructurado en una serie de etapas o **Milestones** principales, que representan los grandes bloques de trabajo y la consecución de objetivos significativos a lo largo del desarrollo del proyecto. Estas fases son clave para organizar el flujo de trabajo y asegurar un progreso coherente. A continuación, se detallan las actividades y entregables esperados en cada una de ellas:

1. **Investigación y análisis acerca de la temática del proyecto:** Esta fase inicial se ha centrado en la exploración profunda del estado del arte, la identificación de tecnologías relevantes y la comprensión del problema a abordar.
 - **Actividades:** Búsqueda de literatura científica, análisis de soluciones existentes, definición de requisitos funcionales y no funcionales de alto nivel.
 - **Entregables:** Notas de investigación, listado inicial de requisitos, primer borrador de la introducción.
2. **Declaración de objetivos a cumplir:** En esta etapa, se han formulado los objetivos específicos, medibles, alcanzables, relevantes y con plazos definidos (SMART) del proyecto.
 - **Actividades:** Redacción de objetivos generales y específicos, justificación de la relevancia del proyecto.
 - **Entregables:** Documento de objetivos, sección de objetivos de la memoria.
3. **Revisión bibliográfica inicial:** Consiste en una profundización de la investigación inicial, con el fin de establecer un marco teórico sólido y contextualizar el trabajo.
 - **Actividades:** Lectura crítica de artículos y publicaciones, identificación de trabajos relacionados, elaboración de un listado de referencias.
 - **Entregables:** Esquema de la revisión bibliográfica, secciones iniciales de la misma.
4. **Desarrollo e implementación técnica del trabajo:** Esta es la fase central del proyecto, donde se ha llevado a cabo la programación y construcción del software.
 - **Actividades:** Diseño de la arquitectura del software, codificación, implementación de algoritmos, desarrollo de módulos.
 - **Entregables:** Código fuente funcional, versiones intermedias del software, documentación interna del código.
5. **Validación técnica del sistema y evaluación de resultados:** En esta etapa se ha comprobado el correcto funcionamiento del sistema y se han analizado los resultados obtenidos.

- **Actividades:** Realización de pruebas unitarias y de integración, análisis de datos, interpretación de resultados, comparación con objetivos.
 - **Entregables:** Informes de pruebas, gráficos y tablas de resultados, sección de resultados de la memoria.
6. **Documentación del proyecto y elaboración de la memoria:** Fase final dedicada a la redacción y consolidación de toda la información del proyecto.
- **Actividades:** Redacción de la memoria del TFG, elaboración de diagramas, revisión y edición final.
 - **Entregables:** Memoria del TFG completa, presentación de defensa.

Planificación detallada: Cronograma o tabla de sprints

La planificación detallada del presente proyecto se ha visualizado a través de un cronograma estructurado, que facilita la comprensión de las fases, tareas y plazos establecidos. El siguiente apartado detalla el cronograma estimado, incluyendo las fases principales, las actividades involucradas y la duración prevista para cada una, con el fin de garantizar una ejecución coherente y eficaz del proyecto dentro del marco académico establecido.

El **Diagrama de Gantt** presentado en la Figura B.1 ofrece una representación visual del cronograma del proyecto. En él, cada barra horizontal representa una tarea o un conjunto de tareas, indicando su duración y las dependencias temporales. Las fases o **Milestones** descritas en la sección anterior están claramente marcadas, permitiendo una visión global de la progresión del trabajo desde el inicio hasta la finalización del TFG. Este diagrama ha servido como una guía fundamental para el seguimiento del progreso y la identificación de posibles cuellos de botella.

La relación entre este cronograma general (más estático) y la planificación ágil (Scrum) se ha gestionado de forma que el Gantt proporciona el marco de alto nivel de las fases principales y los hitos clave, mientras que los *sprints* semanales de Scrum han permitido una planificación más granular y adaptativa de las tareas diarias y semanales dentro de cada gran fase. Esta combinación ha facilitado tanto una visión estratégica del proyecto como una gestión táctica flexible.

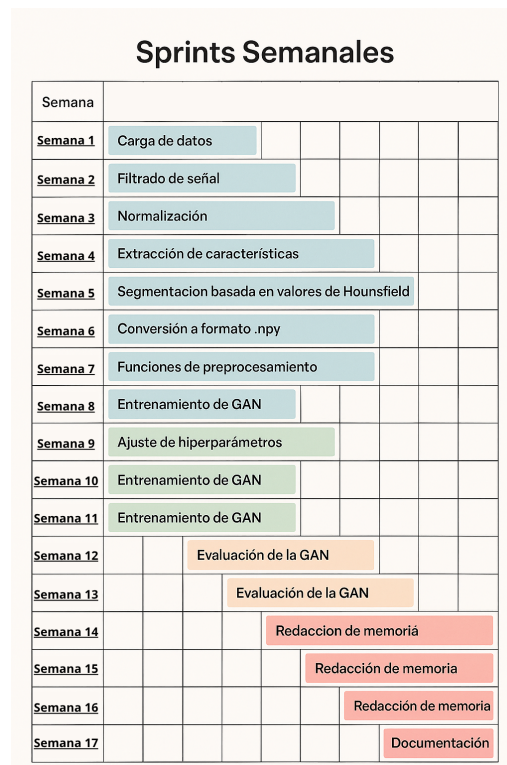


Figura A.1: Organización de las tareas en el tiempo disponible.

Para una visión más detallada de los *sprints* y las tareas específicas, a continuación se presenta una tabla que resume los objetivos y contenidos de cada ciclo semanal:

Seguimiento y control del progreso

El seguimiento y control del progreso ha sido una parte integral de la planificación, asegurando que el proyecto se mantuviera alineado con los objetivos y plazos establecidos. Los mecanismos principales empleados para esta monitorización han sido los siguientes:

- **Reuniones semanales con los tutores:** Como ya se ha detallado, las reuniones de los martes por la tarde han sido el pilar del seguimiento. En ellas, se realizaba una revisión exhaustiva de las tareas completadas en el sprint anterior, se discutían los problemas encontrados y se planificaban las actividades para el siguiente ciclo.

- **Gestión de Issues y Milestones en GitHub:** El uso del sistema de *Issues* y *Milestones* de GitHub ha permitido llevar un registro detallado de las tareas pendientes, en curso y completadas. Cada *issue* representaba una tarea específica, y su asignación a un *milestone* facilitaba la visualización del progreso respecto a los grandes objetivos del proyecto.
- **Control de versiones mediante Git:** El sistema de control de versiones Git, alojado en GitHub, ha permitido un seguimiento continuo del avance en el desarrollo del código. Los *commits* y las *branches* han servido como un registro histórico del trabajo realizado, facilitando la identificación de puntos de progreso y la reversión a estados anteriores si fuera necesario.

En caso de desviaciones o retrasos respecto a la planificación original, la metodología ágil ha facilitado una respuesta rápida. Se han realizado **ajustes en la planificación** durante las reuniones semanales, replanteando la prioridad de ciertas tareas o reasignando esfuerzos para compensar los imprevistos. Este enfoque iterativo y flexible ha sido fundamental para mantener el proyecto en curso y mitigar el impacto de cualquier desafío.

Ventajas y beneficios de la planificación seguida

La planificación adoptada para el desarrollo de este Trabajo de Fin de Grado, basada en la metodología ágil Scrum y adaptada al contexto académico, ha ofrecido numerosos beneficios que han contribuido significativamente al éxito del proyecto:

- **Mejora en la organización y gestión del tiempo:** La división del proyecto en sprints semanales y la definición clara de tareas y objetivos para cada uno han permitido una gestión del tiempo altamente eficiente. Esto ha evitado la procrastinación y ha garantizado un progreso constante y organizado.
- **Garantía de cumplimiento de plazos y calidad:** La revisión continua del avance en las reuniones semanales y la posibilidad de ajustar la planificación sobre la marcha han sido clave para asegurar que los plazos se cumplieran y que el producto final mantuviera un estándar de calidad óptimo.
- **Flexibilidad para adaptarse a cambios o imprevistos:** La naturaleza iterativa de Scrum ha proporcionado una gran flexibilidad. Ante

la aparición de nuevos requisitos, dificultades técnicas inesperadas o ajustes en la disponibilidad, ha sido posible reevaluar las prioridades y adaptar el plan sin comprometer el objetivo general del TFG.

- **Mejor comunicación y coordinación con tutores:** Las reuniones semanales estructuradas han fomentado una comunicación fluida y constante con los tutores. Esto ha facilitado la resolución de dudas, la recepción de feedback oportuno y una mejor coordinación, lo que ha enriquecido el proceso de aprendizaje y el resultado final del proyecto.
- **Visibilidad del progreso:** Las herramientas como GitHub, combinadas con las revisiones semanales, han proporcionado una visibilidad clara del progreso realizado, tanto para el estudiante como para los tutores. Esto ha permitido identificar rápidamente posibles problemas y celebrar los logros intermedios, manteniendo la motivación.

En resumen, la metodología de planificación implementada no solo ha estructurado el desarrollo técnico del TFG, sino que también ha promovido una gestión eficiente, adaptativa y colaborativa, elementos esenciales para el éxito en cualquier proyecto de ingeniería de software.

Apéndice A

Documentación de usuario

Este apéndice proporciona la información necesaria para que otros usuarios puedan comprender, instalar y ejecutar el proyecto desarrollado en este Trabajo de Fin de Grado. Se detallan los requisitos de software y hardware, los pasos para la instalación y puesta en marcha, y una guía práctica para utilizar las funcionalidades principales.

A.1. Requisitos Software y Hardware para Ejecutar el Proyecto

Para poder ejecutar y reproducir el proyecto de generación de señales ECG mediante cWGAN-GP, se requieren los siguientes entornos de software y componentes de hardware:

Requisitos de Hardware

Dado que el proyecto implica el entrenamiento de una red neuronal generativa (GAN), que es computacionalmente intensivo, una Unidad de Procesamiento Gráfico (GPU) es un componente crítico para un rendimiento aceptable.

- **Procesador (CPU):** Se recomienda un procesador multi-núcleo moderno (ej., Intel Core i5/i7/i9 de 8^a generación o superior, o AMD Ryzen 5/7/9).

- **Memoria RAM:** Mínimo 16 GB, aunque se recomiendan 32 GB o más para manejar *datasets* grandes y procesos de entrenamiento complejos.
- **Tarjeta Gráfica (GPU):**
 - **NVIDIA GPU:** Es esencial una GPU compatible con CUDA, con al menos 8 GB de VRAM (ej., NVIDIA GeForce RTX 2060, 3060 o superior, o GPU equivalentes de la serie Quadro/Tesla). Esto es indispensable para un entrenamiento eficiente del modelo.
- **Almacenamiento:** Mínimo 50 GB de espacio libre en disco (SSD recomendado para mayor velocidad) para el código, las dependencias y el almacenamiento de los *datasets* originales y generados.

Requisitos de Software

El proyecto está desarrollado principalmente en **Python** y utiliza bibliotecas de aprendizaje profundo.

- **Sistema Operativo:**
 - Windows 10 (64-bit) o superior.
 - Ubuntu 20.04 LTS (64-bit) o superior (recomendado para desarrollo con GPU).
 - macOS (para ejecución en CPU; el rendimiento con GPU puede variar y requerir configuraciones específicas para Metal o MPS si se usa PyTorch).
- **Python:** Versión 3.8, 3.9 o 3.10. Se recomienda encarecidamente el uso de un gestor de entornos como **conda** (Anaconda/Miniconda) o **venv** para aislar las dependencias del proyecto.
- **Framework de Aprendizaje Profundo:**
 - **PyTorch:** Versión 1.10.0 o superior (con soporte CUDA si se usa GPU).
- **Bibliotecas de Python:** Las dependencias específicas del proyecto se listan en el archivo **requirements.txt**. Las más importantes incluyen:
 - **numpy**
 - **pandas**

- `matplotlib` (para visualización)
 - `scipy` (para procesamiento de señales)
 - `wfdb` (para cargar y manipular los datos del PhysioNet CinC Challenge 2017)
 - `scikit-learn` (para utilidades generales, ej., normalización)
 - `neurokit2` (para análisis de señales ECG, si se incluye su uso)
- **Controladores NVIDIA y CUDA Toolkit** (Solo para GPU NVIDIA):
- Asegúrate de tener los controladores de GPU NVIDIA actualizados.
 - **CUDA Toolkit:** Versión compatible con tu versión de PyTorch (ej., CUDA 11.3, 11.6, 11.8).
 - **cuDNN:** Versión compatible con CUDA Toolkit y PyTorch.

A.2. Instalación / Puesta en Marcha

Esta sección guía al usuario a través del proceso de configuración del entorno y la descarga de los datos para ejecutar el proyecto. Se asume que Python 3 y un gestor de entornos (`conda` o `venv`) ya están instalados.

1. Clonar el Repositorio del Proyecto

Descarga el código fuente del proyecto desde el repositorio de Git (ej., GitHub).

```
git clone [URL_DE_TU_REPOSITORIO]
cd [NOMBRE_DE_TU_CARPETA_PROYECTO]
```

2. Crear y Activar un Entorno Virtual

Es altamente recomendable utilizar un entorno virtual para gestionar las dependencias del proyecto y evitar conflictos con otras instalaciones de Python.

Usando Conda:

```
conda create -n ecg_gan_env python=3.9 % Usa la versión de Python q
conda activate ecg_gan_env
```

Usando venv (Módulo Integrado de Python):

```
python -m venv .venv
source .venv/bin/activate % En Linux/macOS
.\.venv\Scripts\activate % En Windows (PowerShell)
```

3. Instalar las Dependencias del Proyecto

Una vez activado el entorno virtual, instala todas las bibliotecas requeridas listadas en el archivo `requirements.txt`:

```
pip install -r requirements.txt
```

Nota sobre PyTorch con GPU: Si utilizas una GPU, asegúrate de que la instalación de PyTorch incluya el soporte CUDA adecuado. A veces, `pip install -r requirements.txt` se encarga de esto si el `requirements.txt` especifica la versión con CUDA (ej., `torch==2.0.1+cu117`). Si no, puede que necesites instalarlo manualmente siguiendo las instrucciones de su sitio web oficial (ej., `pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu117`).

4. Descargar el Conjunto de Datos (PhysioNet CinC Challenge 2017)

El proyecto utiliza el *dataset* público del PhysioNet/Computing in Cardiology Challenge 2017. Los archivos deben descargarse y colocarse en una estructura de directorio específica.

- Accede a la página del *dataset*: <https://physionet.org/content/challenge-2017/1.0.0/>
- Descarga los archivos `training2017.zip`.
- Descomprime el archivo en la carpeta `data/` dentro del directorio raíz de tu proyecto. Asegúrate de que los archivos `.mat` y `.hea` estén directamente en `data/training2017/`.

La estructura final debería ser algo así:

```
tu_proyecto/
  data/
    training2017/
      A00001.mat
```

```

A00001.he
...
src/
models/
requirements.txt
...

```

A.3. Manuales y/o Demostraciones Prácticas

Esta sección describe cómo utilizar los scripts principales del proyecto para entrenar el modelo, generar señales y realizar evaluaciones.

Estructura de Directorios del Proyecto

Una vez clonado y configurado, el proyecto debería tener una estructura de carpetas similar a:

```

tu_proyecto/
src/           % Contiene el código fuente del modelo y los scripts
  model/       % Definición de la arquitectura de la cWGAN-GP (G y D)
  utils/       % Funciones auxiliares para preprocesamiento, carga de
  train.py     % Script para el entrenamiento del modelo.
  generate.py   % Script para generar señales sintéticas.
  evaluate.py   % Script para la evaluación cuantitativa.
data/          % Directorio para almacenar el dataset original (Phys
models/        % Directorio donde se guardarán los pesos del modelo
output/        % Directorio para resultados de generación, curvas de
config.py      % Archivo de configuración para parámetros del modelo
requirements.txt % Lista de dependencias de Python.

```

Scripts Principales y su Uso

Asegúrate de que estás en la raíz del directorio del proyecto y con el entorno virtual activado antes de ejecutar los comandos.

1. Entrenamiento del Modelo cWGAN-GP (`src/train.py`)

Este script se utiliza para entrenar la red generativa a partir del *dataset* original.

Parámetros configurables: Puedes ajustar parámetros como el número de épocas, el tamaño del lote (*batch size*), las tasas de aprendizaje del generador

Apéndice A

Documentación de usuario

Este apéndice proporciona la información necesaria para que otros usuarios puedan comprender, instalar y ejecutar el proyecto desarrollado en este Trabajo de Fin de Grado. Se detallan los requisitos de software y hardware, los pasos para la instalación y puesta en marcha, y una guía práctica para utilizar las funcionalidades principales.

A.1. Requisitos software y hardware para ejecutar el proyecto

Para poder ejecutar y reproducir el proyecto de generación de señales ECG mediante cWGAN-GP, se requieren los siguientes entornos de software y componentes de hardware:

Requisitos de Hardware

Dado que el proyecto implica el entrenamiento de una red neuronal generativa (GAN), que es computacionalmente intensivo, una ****Unidad de Procesamiento Gráfico (GPU)**** es un componente crítico para un rendimiento aceptable.

- **Procesador (CPU):** Se recomienda un procesador multi-núcleo moderno (ej., Intel Core i5/i7/i9 de 8^a generación o superior, o AMD Ryzen 5/7/9).

- **Memoria RAM:** Mínimo 16 GB, aunque se recomiendan 32 GB o más para manejar *datasets* grandes y procesos de entrenamiento complejos.
- **Tarjeta Gráfica (GPU):**
 - **NVIDIA GPU:** Es esencial una GPU compatible con CUDA, con al menos 8 GB de VRAM (ej., NVIDIA GeForce RTX 2060, 3060 o superior, o GPU equivalentes de la serie Quadro/Tesla). Esto es indispensable para un entrenamiento eficiente del modelo.
- **Almacenamiento:** Mínimo 50 GB de espacio libre en disco (SSD recomendado para mayor velocidad) para el código, las dependencias y el almacenamiento de los *datasets* originales y generados.

Requisitos de Software

El proyecto está desarrollado principalmente en **Python** y utiliza bibliotecas de aprendizaje profundo.

- **Sistema Operativo:**
 - Windows 10 (64-bit) o superior.
 - Ubuntu 20.04 LTS (64-bit) o superior (recomendado para desarrollo con GPU).
 - macOS (para ejecución en CPU; el rendimiento con GPU puede variar y requerir configuraciones específicas para Metal o MPS si se usa PyTorch).
- **Python:** Versión 3.8, 3.9 o 3.10. Se recomienda encarecidamente el uso de un gestor de entornos como **conda** (Anaconda/Miniconda) o **venv** para aislar las dependencias del proyecto.
- **Framework de Aprendizaje Profundo:**
 - **PyTorch:** Versión 1.10.0 o superior (con soporte CUDA si se usa GPU).
- **Bibliotecas de Python:** Las dependencias específicas del proyecto se listan en el archivo **requirements.txt**. Las más importantes incluyen:
 - **numpy**
 - **pandas**

- `matplotlib` (para visualización)
- `scipy` (para procesamiento de señales)
- `wfdb` (para cargar y manipular los datos del PhysioNet CinC Challenge 2017)
- `scikit-learn` (para utilidades generales, ej., normalización)
- `neurokit2` (para análisis de señales ECG, si se incluye su uso)
- **Controladores NVIDIA y CUDA Toolkit** (Solo para GPU NVIDIA):
 - Asegúrate de tener los controladores de GPU NVIDIA actualizados.
 - **CUDA Toolkit**: Versión compatible con tu versión de PyTorch (ej., CUDA 11.3, 11.6, 11.8).
 - **cuDNN**: Versión compatible con CUDA Toolkit y PyTorch.

A.2. Instalación / Puesta en marcha

Esta sección guía al usuario a través del proceso de configuración del entorno y la descarga de los datos para ejecutar el

Apéndice B

Descripción de adquisición y tratamiento de datos

El presente apéndice complementa la información proporcionada en el Capítulo 4: Metodología, específicamente en la sección dedicada a la descripción del conjunto de datos. Con el objetivo de mantener la claridad y fluidez en el cuerpo principal de la memoria, este apartado se destina a la presentación detallada y exhaustiva de las características técnicas y clínicas del dataset utilizado. Por lo tanto, en este apéndice se van a incluir análisis pormenorizados, gráficos adicionales, tablas descriptivas y justificaciones técnicas extensas que, si bien son fundamentales para la reproducibilidad del estudio y para comprender la robustez metodológica, se han omitido del texto principal para facilitar su lectura. Este apéndice proporciona, por tanto, el soporte documental completo sobre la naturaleza y las propiedades del conjunto de datos **PhysioNet / Computing in Cardiology Challenge 2017**. [Goldberger et al., 2000]

B.1. Características generales de los datos

Las principales características de los datos se resumen en el apartado de Metodología de la memoria. Sin embargo, aquí se va a incluir una explicación más detallada acerca del conjunto de datos utilizado.

Tamaño del conjunto de datos

En el momento de la conferencia académica Computing in Cardiology (CinC) el set de datos público presentaba una longitud de 8.528

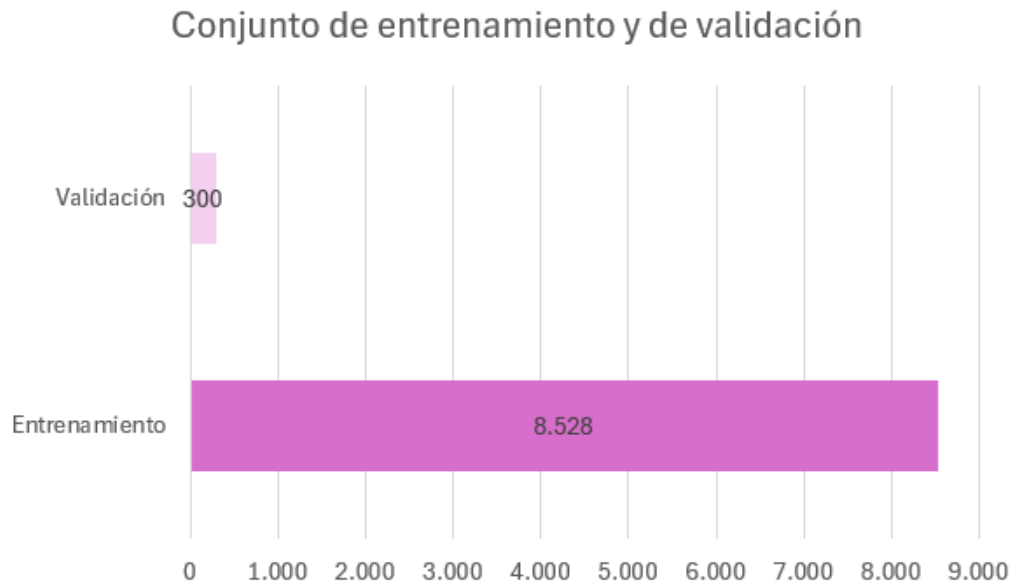


Figura B.1: Gráfico de barras que representa la diferencia de tamaño entre el dataset de entrenamiento y de validación. *Elaboración propia.*

registros de ECG, todos ellos anonimizados y etiquetados clínicamente. [Computer in Cardiology (CinC),] Sin embargo, el tamaño total del conjunto ascendía hasta un total de 12.186 registros electrocardiográficos. Esta ampliación adicional en el número de registros se debe a que había una pequeña parte del conjunto de datos total que no se encontraba pública, por lo que no lo hemos podido utilizar. Por lo tanto, el tamaño de nuestro dataset va a ser de 8.528 registros. [Clifford et al.,]

Sin embargo, estos 8.528 registros estaban etiquetados como "datos de entrenamiento", habiendo otros 300 registros destinados a la validación del funcionamiento de las soluciones creadas a lo largo del desafío. Por lo tanto, estos 300 registros en el presente dataset también se van a destinar a la validación del modelo desarrollado.

Este incremento en el número de señales de validación del conjunto se ha realizado con el propósito de facilitar la realización de tareas de validación externa sobre los algoritmos desarrollados, así como fomentar la reutilización de estos registros como base para futuras investigaciones y desarrollos de modelos. Esta evolución en el número de señales disponibles se puede observar en el gráfico B.1 de la página 24

Cada uno de estos elementos del conjunto de datos corresponde a un registro de ECG individual. Es decir, cada registro corresponde a una monitorización individual y anonimizada realizada mediante un sensor portátil proporcionado por la empresa AliveCor. Por lo tanto, al no contener información identificativa acerca del sujeto sobre el que se realiza la medición, es posible que existan múltiples señales tomadas sobre un mismo paciente en distintos momentos, aunque no se puede asegurar con certeza.

Estos dispositivos portátiles solo realizan mediciones de la actividad eléctrica del corazón en una dirección, por lo que las 12.186 señales que componen el dataset serán de una única derivación, ya que es la configuración predominante en este tipo de dispositivos portátiles diseñados para uso personal y continuo, también conocidos como *wearables*.

Frecuencia de muestreo (fs)

La frecuencia de muestreo es una característica técnica fundamental en el análisis de bioseñales, especialmente en los registros electrocardiográficos (ECG). Determina la resolución temporal con la que la señal continua es digitalizada. Este parámetro, crucial para la fidelidad de la señal, es inherente a la forma en que los datos fueron obtenidos. [Webster, 2020]

Según la documentación técnica oficial del dataset **PhysioNet / Computing in Cardiology Challenge 2017**, la frecuencia de muestreo utilizada durante la adquisición de las señales es de 300 Hz. [Clifford et al.,] Esto significa que se capturaron 300 muestras discretas de la señal analógica por cada segundo de registro. Esta información es vital para nuestro estudio, ya que nos permite determinar con precisión la duración temporal de cada señal, un dato que no se almacena directamente en el dataset, sino que se calcula a partir del número total de muestras y la frecuencia de muestreo.

Esta tasa de muestreo de 300 Hz cumple ampliamente con los requisitos mínimos recomendados por la literatura científica y las directrices clínicas para el análisis de ECG. Por ejemplo, la American Heart Association (AHA) sugiere una frecuencia mínima de 250 Hz para asegurar una representación adecuada de los componentes de alta frecuencia del ECG, permitiendo la identificación fiable de eventos eléctricos clave como las ondas P, el complejo QRS y la onda T [?].

Emplear una frecuencia de muestreo fija y relativamente alta como 300 Hz ofrece varias ventajas metodológicas, las cuales se detallan a continuación:

1. **Fidelidad temporal:** Garantiza una resolución temporal suficiente para capturar las características morfológicas finas y la variabilidad del ritmo cardíaco, algo esencial para la identificación de arritmias.
2. **Homogeneidad de los datos:** Al ser una frecuencia constante en todos los registros, simplifica la normalización y el procesamiento uniforme de los datos, evitando la necesidad de remuestreos adicionales que podrían introducir artefactos o distorsiones.
3. **Eficiencia computacional:** Representa un equilibrio óptimo entre la riqueza de detalle de la señal y el volumen de datos, lo que optimiza la eficiencia computacional durante el preprocesamiento, el entrenamiento del modelo y la evaluación.
4. **Compatibilidad algorítmica:** Facilita la implementación y aplicación de algoritmos de procesamiento digital de señales (como filtros pasa-banda o técnicas de extracción de características), ya que muchos asumen frecuencias de muestreo estandarizadas.

No obstante, es importante reconocer que, si bien una frecuencia de muestreo adecuada es indispensable, no garantiza por sí misma una calidad de señal uniforme. Factores como el ruido inherente a la adquisición en entornos reales, el movimiento del paciente o la calidad del contacto del sensor pueden afectar significativamente la señal capturada. [Luo and Johnston, 2010]

Longitud de las señales

A lo largo de este apartado se detalla la información referente a la longitud de las señales que componen el conjunto de datos. Esta longitud se corresponde con el número de muestras o mediciones que contiene cada una de estas señales electrocardiográficas (ECG). Las señales incluidas en el presente set de datos no presentan una duración homogénea, es decir, no todos los registros tienen el mismo tamaño, puesto que la adquisición se realizó de forma independiente entre unas y otras.

Antes de analizar la duración de cada uno de los registros de ECG que se encuentran en el conjunto de datos, es importante aclarar que la información acerca de la duración de la señal no viene expresada directamente en el dataset. En su lugar, es necesario calcularla a partir del número total de mediciones numéricas que componen cada una de las señales y la frecuencia de muestreo que utiliza el aparato de adquisición de la señal.

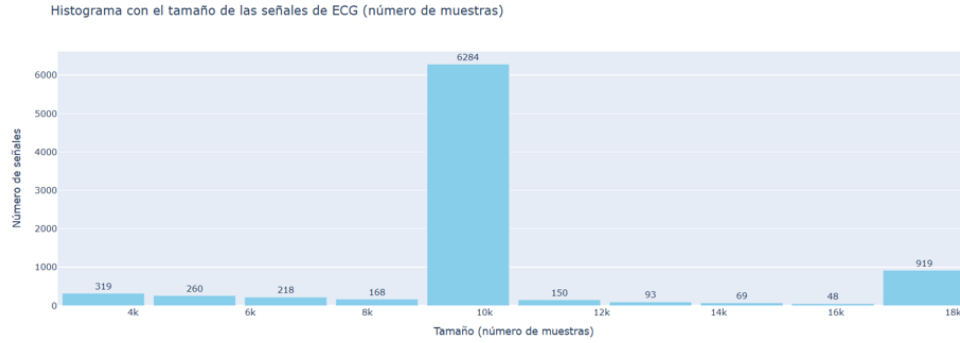


Figura B.2: Histograma que representa el tamaño en número de muestras de las señales de entrenamiento. *Elaboración propia.*

El cálculo de la duración de las señales se va a hacer utilizando la siguiente fórmula:

$$duracionSeñal = \frac{muestrasDataset}{frecuenciaMuestreo} \quad (B.1)$$

En base a la información que recoge la documentación oficial del dataset, las señales están compuestas por entre 2.000 y 20.000 mediciones numéricas, lo que equivale a que la longitud temporal de estas señales oscila entre los 7 segundos y algo más de 60 segundos (1 minuto), aproximadamente. [Clifford et al.,]

En el histograma B.2 de la página 27 podemos observar el número de mediciones por las que se encuentran formadas cada una de las señales del dataset, es decir, el número de muestras que se han tomado a lo largo del proceso de registro de la señal.

A continuación, en el gráfico B.3 de la página 28 se representa la duración de cada una de estas señales, la cual ha sido calculada a partir del número de muestras presentes en cada registro y se expresa en segundos.

Podemos observar en ambas imágenes que esta longitud temporal de los registros está bien calculada, ya que la correspondencia entre el tamaño (número de muestras en cada señal) y la longitud de estas es total. Ambos histogramas presentan el mismo perfil. Además, si sumamos el número total de registros, vemos que asciende a 8.528, valor que se corresponde con el tamaño del dataset público de entrenamiento de la conferencia CinC. [Computer in Cardiology (CinC),]

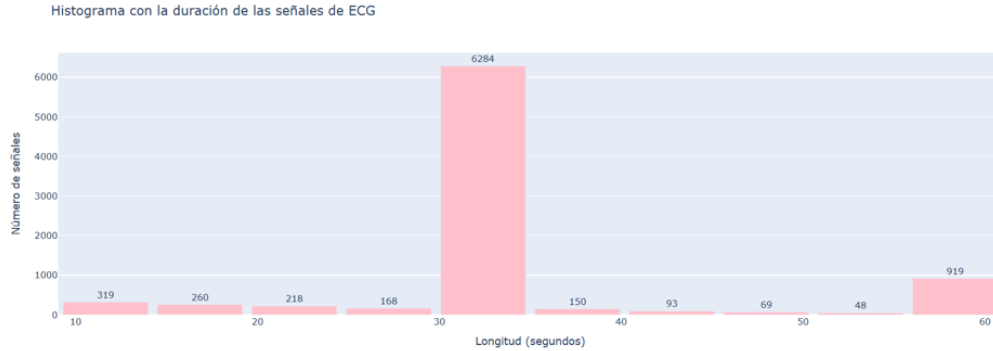


Figura B.3: Histograma que representa la duración en segundos de las señales del dataset de entrenamiento. *Elaboración propia.*

A la hora de interpretar este histograma hay que tener en cuenta que se ha empleado el conjunto de datos de entrenamiento original, es decir, los registros que se usaron durante la conferencia CinC, los cuales ascienden a un total de 8.528 señales de ECG y corresponden a las señales que vamos a utilizar como entrenamiento de nuestro modelo. [Computer in Cardiology (CinC),] Esta representación gráfica nos permite ver que la duración predominante entre las muestras es de un poco más de 30 segundos, con un total de 6.284 muestras. A esta longitud le siguen las señales de ECG con 60 segundos de duración, que son un total de 848 muestras.

Por otra parte, podemos analizar la duración de las señales que se han incorporado al dataset, un total de 300 registros. Este conjunto es el que se va a utilizar como set de datos para la validación del modelo de aprendizaje automático desarrollado a lo largo del proyecto.

Al igual que pasaba con el conjunto de datos de entrenamiento, en la página 29 podemos observar el histograma B.4 en el que se representa el número de muestras por el que se encuentra compuesto cada una de las señales. Mientras que en el histograma B.5 de la página 29 se puede observar la duración de las mismas. En este se ve cómo las longitudes son muy similares entre las muestras del conjunto de entrenamiento y de test, siendo ligeramente inferiores en este último. Sin embargo, esta pequeña diferencia no va a influir sobre el diseño del modelo de aprendizaje automático desarrollado.

Esta duración va a depender de la señal concreta que analicemos y de las condiciones que se siguieron durante su proceso de adquisición. Todas las señales fueron adquiridas en condiciones reales, de carácter clínico o semi-clínico, sin que los pacientes siguieran un patrón homogéneo durante

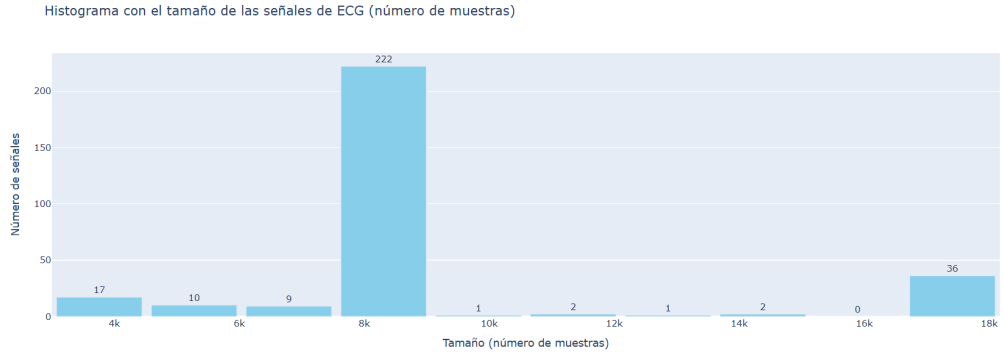


Figura B.4: Histograma que representa el tamaño en número de muestras de las señales de validación. *Elaboración propia.*

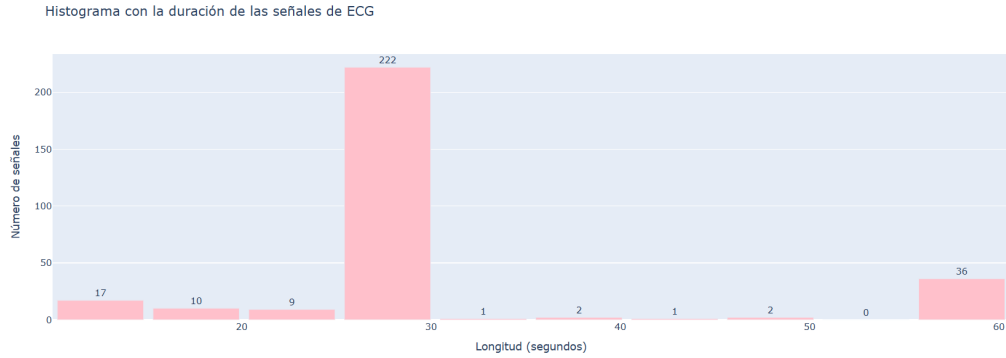


Figura B.5: Histograma que representa la duración en segundos de las señales de validación. *Elaboración propia.*

dicha adquisición. Por lo tanto, en cada caso, la duración va a depender del tiempo durante el que permaneció en funcionamiento el dispositivo de AliveCor, ya que fueron registradas de forma independiente, sin seguir una estandarización estricta acerca del tiempo de registro para cada medición individual. [[AliveCor](#),]

Esta disparidad en la duración de las señales es un foco del dataset sobre el que hay que prestar especial atención, constituyendo un reto técnico importante, tanto en el tratamiento previo como en el análisis de los datos. Esto adquiere una especial relevancia en algunos algoritmos y modelos de aprendizaje automático, como las redes neuronales, que requieren entradas con una longitud determinada, lo que obliga a analizar las características temporales de los registros disponibles. [[Luz et al., 2016](#)]

Para evitar que se vea alterada alguna de las distintas fases del desarrollo del proyecto, desde las etapas de preprocesamiento hasta el diseño del modelo en sí mismo, se lleva a cabo el ya mencionado análisis de las señales. Para evitar este evento, es necesario aplicar técnicas de normalización, segmentación o relleno a las señales. El tipo de estrategia seguida para llevar a cabo estos procesos debe escogerse cuidadosamente ya que puede comprometer la fidelidad temporal de la señal y, por consiguiente, afectar a la capacidad de extracción de patrones por parte del modelo, produciéndose una posible pérdida de información clínica relevante. [Luz et al., 2016]

La duración también puede influir sobre el rendimiento computacional del sistema, limitando la cantidad de información de utilidad que se puede extraer a partir de cada señal o dificultando la detección de eventos cardíacos relevantes (como episodios de fibrilación auricular), los cuales es probable que no se representen en segmentos de longitud menor. [?]

Estas razones fundamentan que el análisis de las señales no sea meramente descriptivo, sino que se trata de un factor de gran importancia a la hora de diseñar el sistema, pudiendo afectar a la toma de decisiones durante el proyecto, tanto a la hora de diseñar el preprocesamiento de los registros como en el momento del diseño de la arquitectura del sistema.

Formato y estructura de los registros

Es imprescindible conocer la estructura interna y el formato de los archivos que componen el conjunto de datos de trabajo, así como la forma en que las señales se encuentran organizadas, para poder llevar a cabo cualquier proceso relacionado bien con el análisis o con el procesamiento de estas. Estas características y propiedades estructurales acerca de los datos se describen en el presente apartado de la memoria, ya que su comprensión resulta esencial para un adecuado tratamiento de los mismos a lo largo de las distintas fases del proyecto: preprocesamiento, segmentación, entrenamiento de la red GAN, etiquetado, entre otras.

Disponer de esta información acerca de los registros permite llevar a cabo una serie de procesos sobre los datos que estos recogen: una correcta interpretación de las señales, una automatización de su carga en el correspondiente entorno de trabajo o incluso una reproducibilidad del flujo de trabajo seguido durante el desarrollo del sistema.

En el caso del presente trabajo, el conjunto de datos utilizado ha sido proporcionado por el **PhysioNet / Computing in Cardiology Challenge 2017** y se encuentra formado por múltiples archivos organizados en base

al formato estándar de PhysioNet. [Goldberger et al., 2000] Esta estructura permite que los datos puedan ser integrados directamente, gracias a bibliotecas como WFDB (WaveForm DataBase), hecho que facilita tanto su manejo como su análisis en entornos como Python o MATLAB. [Xie C and B, 2023]

Siguiendo esta estructura, cada registro individual de ECG se encuentra formado por una pareja de archivos, que comparten el mismo nombre pero difieren en la extensión:

- **Archivo con extensión .hea:** Este archivo *header* se trata de un archivo de texto plano y contiene la información auxiliar del archivo, la cual es necesaria para su correcta lectura, interpretación y procesamiento. Este contiene metadatos como la frecuencia de muestreo, el número de muestras que componen cada registro, el número de derivaciones (en este caso, una) o la etiqueta diagnóstica correspondiente.
- **Archivo con extensión .mat:** Este archivo almacena la señal cruda del electrocardiograma en formato binario, es decir, contiene los valores numéricos que conforman la señal recogida por el dispositivo (electrocardiógrafo) en formato de MATLAB. Estos se encuentran codificados en forma de matriz unidimensional, ya que las señales presentan únicamente una derivación, así como la clase a la que pertenece cada una de las señales. Por lo tanto, la presencia de la clase en este archivo es uno de los puntos fuertes de dataset, ya que le está concediendo la funcionalidad de ser usado en tareas de clasificación.

Apéndice A

Estudio experimental

En el presente anexo se recoge el conjunto de experimentos y pruebas realizados a lo largo del desarrollo del Trabajo de Fin de Grado (TFG).

Este trabajo se ha centrado en torno a la generación sintética de señales electrocardiográficas (ECG) a través del uso de redes generativas adversariales (GAN). Por lo tanto, las distintas pruebas ensayadas a lo largo de su desarrollo y documentadas en este anexo están relacionadas con dicha temática. Estos procesos abarcan aspectos como las distintas configuraciones aplicadas a lo largo del preprocesamiento y la limpieza de los datos, las arquitecturas de red exploradas, así como las técnicas y parámetros de entrenamiento testeados con el objetivo de obtener señales sintéticas de calidad, las cuales sean representativas de las clases reales.

El contenido del anexo acerca de las distintas configuraciones y pruebas se organiza en tres secciones principales, las cuales se van a detallar a continuación.

En primer lugar, se encuentra el cuaderno de trabajo, apartado en el que se enumeran los distintos métodos y enfoques probados a lo largo del proyecto. En este se incluyen tanto los que ofrecieron resultados satisfactorios como aquellos que fueron descartados. Esto se debe a que todos, tanto los positivos como los negativos, contribuyen a la toma de decisiones conscientes y documentadas, al ajuste progresivo del modelo y al proceso de aprendizaje.

A continuación, se describe la configuración final utilizada, la que se podría destacar como *configuración ganadora*. Entre los detalles ofrecidos acerca de la misma se pueden destacar los valores utilizados para los distintos parámetros y argumentos en cada fase, justificando las elecciones adoptadas

en base a los resultados de configuraciones o fases previas. A su vez, también se incluyen los motivos que han promovido dicha elección, así como los detalles técnicos más relevantes para la reproducción del experimento.

Por último, se encuentra el apartado dedicado a la ampliación de los resultados obtenidos a lo largo del desarrollo del proyecto, incluyendo tanto un resumen visual como cuantitativo de los mismos. Este permite evaluar el rendimiento de la arquitectura de red implementada y la calidad de las señales generadas.

Por lo tanto, este apéndice, en conjunto, tiene como finalidad aportar transparencia al proceso experimental, justificando y documentando las decisiones técnicas adoptadas y facilitando la reproducibilidad del trabajo realizado en ensayos futuros.

A.1. Cuaderno de trabajo

En esta sección se documenta de forma cronológica o comparativa los diversos experimentos y pruebas realizados durante el desarrollo del proyecto. Se detallan tanto los enfoques que resultaron exitosos como aquellos que fueron descartados, junto con una breve justificación de los resultados observados. La transparencia de este cuaderno de bitácora.^{es} crucial para entender el proceso iterativo de diseño y optimización del modelo.

Apéndice B

Plan de Proyecto Software

B.1. Introducción

Antes de iniciar un proyecto, especialmente uno relacionado con el desarrollo de software, es fundamental establecer una adecuada planificación. Esto responde a la propia naturaleza de esta disciplina, ya que constituye una actividad integral. Por ello, no requiere únicamente de competencias técnicas, sino que también incluye aspectos de gestión, evaluación y cumplimiento normativo. [Sommerville, 2015]

Esta tarea resulta clave incluso cuando se trabaja en el ámbito académico, como en un Trabajo de Fin de Grado. Con este objetivo, el presente trabajo se ha realizado siguiendo los principios básicos de la Ingeniería del Software y la gestión de proyectos tecnológicos, utilizando un enfoque técnico y riguroso. [Basili, 2006]

Con este propósito, el plan de desarrollo que se presenta a continuación busca organizar, estructurar, dimensionar e incluso justificar las diferentes fases del proyecto. Para ello, se contemplan aspectos y estrategias a nivel técnico, temporal, logístico, económico y legal, garantizando así su viabilidad y coherencia. Además, permite evitar complicaciones una vez iniciado el proyecto.

Este Plan de Proyecto Software constituye un documento de referencia que plasma de forma clara y estructurada los factores determinantes relacionados con la gestión, asignación de recursos y desarrollo del proyecto, complementando el enfoque teórico y técnico descrito en el cuerpo principal de la memoria.

Este apéndice se organiza en tres secciones fundamentales, que se detallan a continuación, y que se resumen en la tabla B.1, situada en la página 36.

Sección	Objetivos	Contenido
Planificación Temporal	Organizar el proyecto a lo largo de su longitud en el tiempo	Cronograma, fases y hitos
Planificación Económica	Presentar la inversión necesaria y la viabilidad financiera	Presupuesto, recursos y coste
Viabilidad Legal	Garantizar la legalidad, cumpliendo la normativa vigente	Normativas, licencias y aspectos regulatorios

Tabla B.1: Presentación de las secciones del Plan de Proyecto. *Elaboración propia*

B.2. Planificación temporal

La planificación temporal es fundamental en la gestión eficaz de proyectos. [(PMI), 2021] Su importancia radica en la necesidad de establecer un marco de trabajo organizado y estructurado que permita la distribución sistemática de tareas, asegurando la finalización exitosa de las actividades dentro de los plazos previstos. En el presente proyecto, esta organización temporal ha sido clave. [Sommerville, 2015]

Esta dimensión temporal adquiere un valor añadido en proyectos técnicos o de investigación, especialmente los de desarrollo de software. Esto se debe a su naturaleza multifacética y a la estructuración en fases iterativas de diversa índole, cada una con un nivel de complejidad, requisitos y duración particular.

Una planificación adecuada facilita la consecución de objetivos en los plazos establecidos, permite anticiparse a posibles problemas, optimiza el uso de recursos y mejora la toma de decisiones, evitando retrasos y asegurando una calidad óptima del producto final.

En contextos académicos, una planificación temporal correcta es fundamental. Garantiza que el proyecto se desarrolle de forma coherente, alineándose con el calendario académico y los requisitos formales de la titulación, asegurando así un progreso bien documentado.

Para este TFG, se ha optado por una planificación iterativa basada en la metodología ágil ****Scrum****, adaptada al contexto académico mediante modificaciones como la división del trabajo en sprints semanales y revisiones periódicas con los tutores. [Schwaber, Ken and Sutherland, Jeff,]

La planificación se organiza en una secuencia lógica de fases o etapas de una semana de duración (*sprints*), donde se establecen tareas u objetivos concretos. Dichas tareas son revisadas semanalmente en una reunión programada con los tutores. En estas reuniones se valida el trabajo realizado, se resuelven dudas y se planifican los hitos para la semana en curso.

Esta distribución temporal del TFG se visualiza en la imagen B.1, que representa un diagrama de Gantt. En este cronograma se muestran de forma clara las actividades previstas, su duración, dependencias temporales y los objetivos o hitos intermedios. Por lo tanto, el diagrama sirve como guía orientativa de plazos y como herramienta para evaluar continuamente el progreso, gestionando eficazmente la distribución de tareas.

A grandes rasgos, las tareas se agrupan en las siguientes etapas o **Milestones**:

Investigación y análisis de la temática del proyecto	Declaración de objetivos a cumplir	Revisión bibliográfica inicial	Desarrollo e implementación técnica del trabajo	Validación técnica del sistema y evaluación de resultados	Documentación del proyecto y elaboración de la memoria
--	------------------------------------	--------------------------------	---	---	--

Cabe señalar que algunas etapas, como la investigación preliminar, la definición de objetivos o la elaboración de la memoria, no forman parte de la sección de Metodología, ya que no corresponden directamente con las fases técnicas o científicas. No obstante, son tareas complementarias fundamentales para garantizar una ejecución completa y rigurosa del TFG.

Metodología de la planificación

La metodología utilizada para la organización de esta planificación ha sido la metodología ágil **Scrum**, adaptada al entorno académico. Este marco ágil es ampliamente usado en el desarrollo de software, considerándose un *gold standard* por muchos profesionales. [Schwaber, Ken and Sutherland, Jeff,]

Scrum destaca por su organización iterativa e incremental del trabajo, dividiendo este en bloques cortos y repetitivos (*sprints*). Esto permite definir hitos clave o entregables intermedios y equilibrar cargas de trabajo y recursos disponibles, garantizando un desarrollo ordenado y eficiente.

Este enfoque presenta numerosas ventajas, como una respuesta rápida y flexible ante cambios, la evaluación continua de los avances y la mejora progresiva de las distintas versiones del producto final (el TFG). En definitiva, garantiza una correcta gestión del trabajo.

Adaptación de la metodología Scrum al entorno académico

Dado que este Trabajo de Fin de Grado no se ha desarrollado en un entorno empresarial con un equipo de desarrollo completo, sino de forma individual bajo la supervisión de dos tutores, se han realizado las siguientes adaptaciones al modelo clásico de Scrum:

- **Estructura en sprints semanales:** El trabajo se ha organizado en ciclos de una semana, lo que ha permitido un control detallado del avance y una carga de trabajo equilibrada.
- **Roles adaptados:** El estudiante ha asumido un rol combinado de "equipo de desarrollo" y "Product Owner". Los tutores han desempeñado un papel similar al de "Scrum Master", facilitando el proceso y resolviendo impedimentos.
- **Reuniones de revisión y planificación semanales:** Todos los martes por la tarde se ha llevado a cabo una reunión con los tutores. En estas sesiones se revisaban los avances del sprint anterior (*Sprint Review*), se resolvían dudas y se establecían los objetivos para el sprint siguiente (*Sprint Planning*).
- **Herramientas de gestión simplificadas:** El repositorio de **GitHub** ha sido la plataforma principal para el control de versiones del código y la organización de **Milestones** e **Issues**, funcionando como un *Sprint Backlog* rudimentario. Un **calendario** personal y los acuerdos en reuniones complementaron la gestión.
- **Entregables continuos:** Cada sprint ha generado avances en el desarrollo técnico y en la documentación del proyecto, permitiendo una evaluación continua del progreso.

Estas adaptaciones han permitido aprovechar los beneficios de la agilidad de Scrum, como la flexibilidad, la transparencia y la mejora continua, dentro de las características del entorno académico.

Etapas o Milestones del proyecto

El TFG se ha estructurado en etapas o **Milestones**, que representan los grandes bloques de trabajo y la consecución de objetivos significativos.

1. **Investigación y análisis:** Exploración profunda del estado del arte, identificación de tecnologías y comprensión del problema.

2. **Declaración de objetivos:** Formulación de objetivos específicos, medibles, alcanzables, relevantes y con plazos definidos (SMART).
3. **Revisión bibliográfica:** Profundización en la investigación para establecer un marco teórico sólido y contextualizar el trabajo.
4. **Desarrollo e implementación técnica:** Fase central de programación y construcción del software.
5. **Validación técnica y evaluación:** Comprobación del funcionamiento del sistema y análisis de los resultados obtenidos.
6. **Documentación y memoria:** Redacción y consolidación de toda la información del proyecto.

Planificación detallada: Cronograma o tabla de sprints

La planificación detallada se visualiza a través de un cronograma estructurado. El **Diagrama de Gantt** en la Figura B.1 ofrece una representación visual del cronograma, mostrando duración y dependencias de las tareas, y marcando las fases o **Milestones**. Ha servido como guía y herramienta de evaluación.

La combinación del cronograma general (marco de alto nivel) con los sprints semanales de Scrum (planificación granular) ha facilitado tanto una visión estratégica como una gestión táctica flexible.

Seguimiento y control del progreso

El seguimiento y control del progreso ha sido esencial. Los mecanismos principales empleados han sido:

- **Reuniones semanales con los tutores:** Pilar del seguimiento, con revisión de tareas, discusión de problemas y planificación.
- **Gestión de Issues y Milestones en GitHub:** Registro detallado de tareas y visualización del progreso.
- **Control de versiones mediante Git:** Seguimiento continuo del avance del código, con *commits* como registro histórico.

En caso de desviaciones, la metodología ágil ha permitido una respuesta rápida con ajustes en la planificación, replanteando prioridades o reasignando esfuerzos para compensar imprevistos.

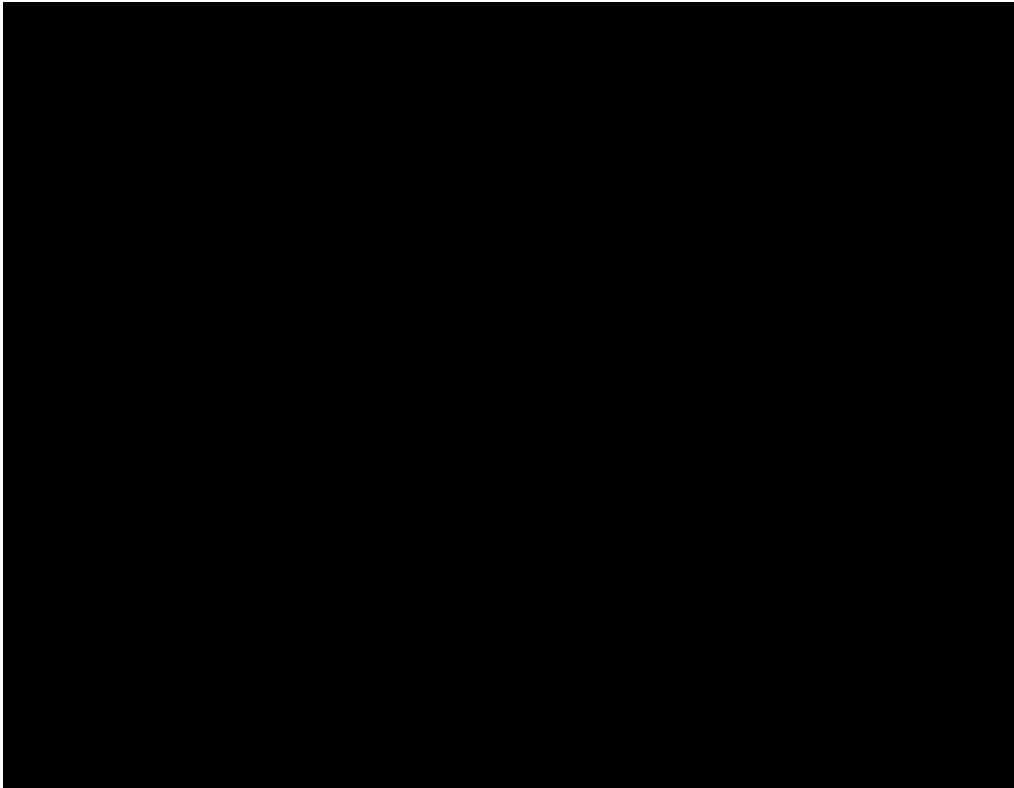


Figura B.1: Diagrama de Gantt del Plan de Proyecto. *Elaboración propia*

Ventajas y beneficios de la planificación seguida

La planificación adoptada ha ofrecido numerosos beneficios:

- **Mejora en la organización y gestión del tiempo:** Sprints semanales y definición clara de tareas han permitido una gestión eficiente, evitando la procrastinación.
- **Garantía de cumplimiento de plazos y calidad:** La revisión continua y la adaptación flexible han asegurado el cumplimiento de plazos y un alto estándar de calidad.
- **Flexibilidad para adaptarse a cambios o imprevistos:** La naturaleza iterativa de Scrum ha proporcionado gran flexibilidad ante nuevos requisitos o dificultades.

- **Mejor comunicación y coordinación con tutores:** Las reuniones semanales han fomentado una comunicación fluida, facilitando la resolución de dudas y el feedback.
- **Visibilidad del progreso:** Herramientas como GitHub y las revisiones semanales han proporcionado visibilidad clara, ayudando a identificar problemas y mantener la motivación.

En resumen, la planificación implementada ha estructurado el desarrollo técnico y ha promovido una gestión eficiente, adaptativa y colaborativa, elementos esenciales para el éxito en cualquier proyecto de ingeniería de software.

Bibliografía

- [AliveCor,] AliveCor. AliveCor española. [Internet, accedido el 25 de abril de 2025].
- [Basili, 2006] Basili, V. R. (2006). The past, present, and future of experimental software engineering. *Journal of The Brazilian Computer Society*, 12(3):7 – 12.
- [Clifford et al.,] Clifford, G. D., Liu, C., Moody, B., Lehman, L.-w. H., Silva, I., Li, Q., Johnson, A. E., and Mark, R. G. Af classification from a short single lead ecg recording: the physionet/computing in cardiology challenge 2017. [Internet, accedido el 24 de abril de 2025].
- [Computer in Cardiology (CinC),] Computer in Cardiology (CinC). Computer in cardiology (cinc). [Internet, accedido el 25 de abril de 2025].
- [Goldberger et al., 2000] Goldberger, A., Amaral, L., Glass, L., Hausdorff, J., Ivanov, P. C., Mark, R., and Stanley, H. E. (2000). Physiobank, physiotoolkit, and physionet: Components of a new research resource for complex physiologic signals. *Circulation (Online)*, 101(23):e215 – e220. RRID:SCR₀07345.
- [Luo and Johnston, 2010] Luo, S. and Johnston, P. (2010). A review of electrocardiogram filtering. *Journal of Electrocardiology*, 43:486 – 496.
- [Luz et al., 2016] Luz, E. J. d. S., Schwartz, W. R., Cámara-Chavez, G., and Menotti, D. (2016). Ecg-based heartbeat classification for arrhythmia detection: A survey. *Computer Methods and Programs in Biomedicine*, 127:144 – 164.

- [(PMI), 2021] (PMI), P. M. I. (2021). *A guide to the Project Management Body of Knowledge (PMBOK guide) and the Standard for project management*. Project Management Institute.
- [Schwaber, Ken and Sutherland, Jeff,] Schwaber, Ken and Sutherland, Jeff. La guía scrum. la guía definitiva de scrum: Las reglas del juego. [Internet, accedido el 15 de marzo de 2025].
- [Sommerville, 2015] Sommerville, I. (2015). *Software Engineering*. Pearson, 10 edition.
- [Webster, 2020] Webster, J. G. (2020). *Medical Instrumentation: Application and Design*. Wiley, 5 edition.
- [Xie C and B, 2023] Xie C, McCullum L, J. A. P. T. G. B. and B, M. (2023). Waveform database software package (wfdb) for python (version 4.1.0). [PhysioNet, versión 4.1.0].